B.TECH CCE REVISION · PART 10

Programming Basics for Reading and Reviewing Code

Covers the marksheet subjects Problem Solving Using Computers, Object Oriented Programming, Data Structures, Design and Analysis of Algorithms, and their labs.

The revision map deliberately skipped problem-grinding: AI now writes much of the code. What remains essential is being able to **read** code, judge whether it's sensible, spot when it will be slow or wrong, and tell an AI (or an engineer) precisely what to change. That needs the vocabulary and a feel for costs, not the ability to invent algorithms under exam pressure.

Enough programming to read, review and direct AI-written code: how programs are structured, object-oriented design, data structures and what they cost, Big-O intuition, recursion, the common algorithms, and a checklist for reviewing code.

Examples are in Python because it's the easiest to read; the ideas carry over to JavaScript, Java and the rest.

How to use this chapter

> **How to read it.** Read with a Python prompt open and run the snippets. The labs are mostly "read this code and predict what it does", the skill that matters most when reviewing AI output.

Isagi · November 2026 · Written from general knowledge of the field as of late 2026; check tool versions, prices and vendor specifics against current documentation.

Contents

1. How programs are built
2. Types, values and references
3. Functions, scope and errors
4. Object-oriented programming
5. Data structures and what they cost
6. Big-O intuition
7. Recursion
8. Algorithms worth recognising
9. Reading and reviewing code
10. Labs
11. Self-check questions and answers
12. Resources

1. How programs are built

A program is data plus instructions that transform it, organised into small named pieces so humans can understand and change it.

- **Building blocks** in every language: values and variables; expressions; **conditionals** (`if`); **loops** (`for`, `while`); **functions**; **collections** (lists, dictionaries); and **modules** to group related code into files.
- **Compiled vs interpreted**: compiled languages (C, Go, Rust) are translated to machine code ahead of time, which is fast and catches many errors early; interpreted or JIT languages (Python, JavaScript) run through a runtime, which is flexible and slower. Java and C# compile to bytecode for a virtual machine.
- **Static vs dynamic typing**: static languages (Java, TypeScript, Go) check types before running; dynamic ones (Python, JavaScript) check while running. Type hints in Python and TypeScript over JavaScript bring many static benefits, and they make AI-generated code easier to check.
- **Paradigms**: **procedural** (step-by-step functions), **object-oriented** (objects bundling data and behaviour, Section 4), **functional** (pure functions, immutable data, transforming collections with map/filter/reduce). Modern code mixes all three.
- **The ecosystem matters as much as the language**: package managers (pip, npm), libraries, frameworks (Django/FastAPI, React/Next.js), linters, formatters and test runners.

2. Types, values and references

Many confusing bugs come from not knowing whether two names point at the same thing.

- **Primitive-like values** (numbers, strings, booleans) behave as if copied. **Mutable objects** (lists, dicts, most objects) are shared by reference: assigning one to another name doesn't copy it.

```
a = [1, 2, 3]
b = a          # b is another name for the same list
b.append(4)
print(a)       # [1, 2, 3, 4]  -- a changed too
c = a.copy()   # a new list
```

- **Mutability**: strings and tuples can't be changed in place; lists and dicts can. Immutable data is easier to reason about, which is why functional styles prefer it.
- **Equality vs identity**: `==` compares values; `is` checks if two names point at the same object.
- **Floating point is approximate**: `0.1 + 0.2 == 0.3` is `False` in almost every language, because binary can't represent 0.1 exactly. **Never store money as floats**; use integer paise or a decimal type.
- **None / null**: the absence of a value, and the source of the most common crash ("NoneType has no attribute..."). Check for it at boundaries.
- **Strings and encodings**: text is Unicode; on disk and over the network it's bytes, usually UTF-8. Mixing them up garbles Hindi, Malayalam and emoji.

3. Functions, scope and errors

Small functions with clear inputs and outputs are the unit of readable, testable code.

- A good function **does one thing**, has a name that says what, takes inputs as parameters, and returns a result rather than changing global state. **Pure functions** (same input, same output, no side effects) are the easiest to test and trust.
- **Scope**: variables live in the block or function where they're defined. Globals shared across the program cause surprising interactions.
- **Default argument trap** (Python): `def add(item, items=[])` reuses the same list on every call. Use `items=None` and create a new list inside.
- **Errors and exceptions**: when something goes wrong, code **raises** an exception; callers **catch** the ones they can handle. Warning signs in review:
- `except: pass` or `catch (e) {}` (swallowing errors silently, so failures become invisible).
- Catching everything instead of specific errors.
- Returning `None` or `-1` on failure without the caller checking.
- **Validate at the boundaries** (user input, API responses, files), then trust data inside.
- **Async functions** (`async def`, `await`) let a program wait on many network calls at once (Part 3). A missing `await` is a classic bug: the call silently never runs or returns a pending object.

4. Object-oriented programming

Objects bundle data with the functions that operate on it, so related things change together and the rest of the program uses a simple interface.

Concept	Meaning	Example
Class	A blueprint	`Session`
Object / instance	A thing built from it	one particular practice session
Attributes	Data held by the object	`user_id`, `duration_s`, `transcript`
Methods	Functions that belong to it	`session.word_count()`
Encapsulation	Hide internals; expose a small interface	callers use `score()`, not the scoring internals
Inheritance	A class extends another, reusing and overriding	`DebateSession(Session)`
Polymorphism	Different classes respond to the same method in their own way	every rubric has `.score(answer)`
Abstraction / interface	Define *what* something does, not *how*	an `LLMClient` interface with several providers

```
from dataclasses import dataclass
from typing import Protocol

class Rubric(Protocol):              # an interface: anything with .score()
    name: str
    def score(self, transcript: str) -> float: ...

@dataclass
class FillerWords:
    name: str = "fillers"
    fillers: tuple = ("um", "uh", "like", "basically", "actually")
    def score(self, transcript: str) -> float:
        words = transcript.lower().split()
        rate = sum(w.strip(",.") in self.fillers for w in words) / max(len(words), 1)
        return round(max(0.0, 10 - rate * 100), 1)   # 10% fillers -> 0

@dataclass
class Length:
    name: str = "length"
    target_words: int = 150
    def score(self, transcript: str) -> float:
        n = len(transcript.split())
        return round(10 * min(n, self.target_words) / self.target_words, 1)

def score_all(transcript: str, rubrics: list[Rubric]) -> dict[str, float]:
    return {r.name: r.score(transcript) for r in rubrics}   # polymorphism

print(score_all("Um, basically I think, like, teamwork matters", [FillerWords(), Length()]))
# {'fillers': 0.0, 'length': 0.5}  -- 3 fillers in 7 words, and far too short
```

Practical OOP judgement (and what to look for in AI-written code):

- **Prefer composition over deep inheritance.** Hierarchies three or four levels deep become hard to change.
- **Small interfaces** (`score(transcript)`) let you swap implementations, including fakes in tests.

- Not everything needs a class. A plain function or a data record (`@dataclass`, a TypeScript type) is often clearer. AI tools sometimes over-engineer with factories, managers and base classes for one use.

5. Data structures and what they cost

Choosing the right container is the single biggest everyday performance decision. Each one makes some operations fast and others slow.

Structure	Python / JS	Fast at	Slow at	Use for
Array / list	`list` / `Array`	Index access O(1), append O(1)	Search by value O(n), insert/remove at front O(n)	Ordered sequences
Hash map	`dict` / `Map`, object	Lookup, insert, delete by key: O(1) average	Ordered or range queries	Lookups by ID, counting, caching, grouping
Set	`set` / `Set`	Membership test O(1)	Ordering	De-duplication, "have we seen this?"
Linked list	(rarely used directly)	Insert/remove at a known node O(1)	Index access O(n)	Queues, LRU caches (inside libraries)
Stack (LIFO)	`list` with append/pop	Push/pop O(1)	Anything else	Undo, parsing, function calls, DFS
Queue (FIFO)	`collections.deque`	Add at back, remove at front O(1)	Random access	Task queues, BFS
Heap / priority queue	`heapq`	Get smallest/largest O(1), insert O(log n)	Search	Top-k, scheduling, Dijkstra
Balanced tree / B-tree	(in databases, sorted containers)	Search, insert, range queries O(log n)	—	Database indexes (Part 4), sorted data
Graph	dict of lists	Representing relationships	—	Networks, dependencies, routes, social links
Trie	(libraries)	Prefix lookup	Memory	Autocomplete

The most common performance bug in real code, AI-written or not: **checking membership in a list inside a loop.**

```
emails = [...]           # 100,000 registered emails
new = [...]              # 100,000 sign-ups to check
dupes = [e for e in new if e in emails]        # list lookup: ~10 billion comparisons
email_set = set(emails)
dupes = [e for e in new if e in email_set]     # set lookup: ~100,000 hash lookups
```

Same result; the first takes minutes, the second milliseconds.

Hash maps are the workhorse: a **hash function** turns a key into an array position, so lookup doesn't depend on size. Collisions (two keys landing in one slot) are handled internally. Keys must be immutable (strings, numbers, tuples).

6. Big-O intuition

Big-O describes how work grows as input grows. You don't need proofs; you need to recognise the shapes.

Big-O	Name	Example	n = 1,000 → 1,000,000
O(1)	Constant	Dict lookup, array index	Same time
O(log n)	Logarithmic	Binary search, B-tree index lookup	~10 → ~20 steps
O(n)	Linear	One loop over the data	1,000× more time
O(n log n)	Linearithmic	Good sorting	~2,000× more
$O(n^2)$	Quadratic	Loop inside a loop over the same data; list membership inside a loop	1,000,000× more
$O(2^n)$	Exponential	Trying every subset	Hopeless beyond ~40 items

How to read it in code:

- **One loop over n items**: O(n). **A loop inside a loop** over the same data: $O(n^2)$. This is the shape to catch in review.
- **Hidden loops**: `x in list`, `list.index()`, `list.remove()`, string `+=` in a loop, and database queries inside loops (the N+1 problem, Part 4) are loops even though they don't look like it.
- Big-O ignores constants: an O(n) step that makes a network call per item is still much slower than an O(n log n) step in memory. **I/O dominates**: one avoidable database or API call usually costs more than all the in-memory work around it.
- **Space** matters too: loading a 5 GB file into memory to count lines is O(n) space when streaming would be O(1).
- **When it matters**: below a few thousand items, almost anything is fast. Big-O matters when data grows (users, sessions, logs) or when code runs on every request.

7. Recursion

A function that solves a problem by calling itself on a smaller version of the same problem, until it reaches a case simple enough to answer directly.

```
def count_files(folder: dict) -> int:
    """folder looks like {"a.txt": None, "sub": {"b.txt": None}}"""
    total = 0
    for name, child in folder.items():
        if child is None:            # base case: a file
            total += 1
        else:                         # recursive case: a sub-folder
            total += count_files(child)
    return total

print(count_files({"a.txt": None, "sub": {"b.txt": None, "deeper": {"c.txt": None}}}))  # 3
```

- Every recursive function needs a **base case** that stops it, and each call must move toward it. Otherwise: infinite recursion and a **stack overflow** (Part 3).
- Recursion fits naturally **recursive data**: folders, comment threads, JSON, organisation charts, the HTML DOM, expression trees.
- Each call uses stack memory; very deep recursion (Python's default limit is about 1,000) can crash. Loops or explicit stacks avoid that.
- **Memoisation**: cache results of calls already made. Naive recursive Fibonacci is $O(2^n)$; with a cache (`functools.cache`) it's O(n). That's the core idea of **dynamic programming**: break a problem into overlapping subproblems and reuse their answers.

8. Algorithms worth recognising

You won't implement these, but you should know they exist, what they cost and when code is reinventing one badly.

- **Searching**: linear search O(n); **binary search** O(log n) on sorted data (halve the range each step: how you find a word in a dictionary).
- **Sorting**: good general sorts are O(n log n) (merge sort, quicksort; Python's built-in Timsort). Never hand-write a sort in production; call the built-in with a `key`.
- **Hashing** for de-duplication, grouping and counting in O(n).
- **Two pointers / sliding window**: process a sequence once while tracking a range, e.g. the busiest 7-day window.
- **Graph traversal**: **BFS** (breadth-first, finds shortest paths in unweighted graphs: degrees of separation) and **DFS** (depth-first, explores deeply: dependency resolution, maze solving). **Dijkstra** finds shortest paths with weights (maps, routing). **Topological sort** orders tasks with dependencies (build systems, course prerequisites).
- **Greedy algorithms**: take the best-looking choice at each step; sometimes optimal (making change with Indian denominations), sometimes not.
- **Dynamic programming**: reuse overlapping subproblem results (edit distance between two strings, used in spell-check and in comparing transcripts to reference answers).
- **Divide and conquer**: split, solve halves, combine (merge sort, binary search).
- **Hard problems**: some problems (optimal delivery routes for many stops, perfect scheduling) have no known fast exact algorithm (NP-hard); real systems use heuristics and approximations that are good enough.

9. Reading and reviewing code

Whether it was written by a person or an AI, review for correctness, clarity, safety and cost, in that order.

How to read unfamiliar code: start from the entry point (the route handler, `main`, the button's click handler) and follow one real request through; read the tests, which show intended behaviour; read function names and signatures before bodies; run it and add a print or breakpoint where you're unsure.

Review checklist

1. **Does it do what was asked?** Check against the acceptance criteria, including edge cases: empty input, one item, huge input, missing fields, duplicate submissions, non-English text, slow or failing network.
2. **Errors**: are failures handled or at least surfaced, not swallowed? Are timeouts set on network calls?
3. **Security** (Part 6): user input reaching SQL, shell commands or HTML; ownership checks on every object access; no secrets in code; no sensitive data in logs.
4. **Cost and performance**: loops inside loops, membership tests on lists, queries or API calls inside loops, loading everything into memory, missing pagination.
5. **Concurrency** (Part 3): shared mutable state, read-modify-write races, missing `await`.
6. **Clarity**: names that say what things are; functions short enough to understand; no clever tricks where plain code would do; comments explaining *why*, not *what*.
7. **Tests**: do they exist, do they test behaviour rather than implementation details, and would they fail if the code were wrong?
8. **Fit**: does it follow the project's structure and conventions, or invent a parallel way of doing something that already exists? AI tools often duplicate existing helpers.
9. **Scope**: does the change include unrelated edits? Small, focused changes are easier to review and revert.

AI-specific red flags: calls to library functions or API parameters that don't exist (hallucinated APIs); confident comments that don't match the code; tests that were edited to pass rather than code fixed; catch-all error handling added to make an error disappear; and large rewrites when a small fix was asked for.

Git basics every reviewer needs: a **commit** is a saved snapshot with a message; a **branch** is a line of work; a **pull request** proposes merging a branch, with a diff to review; `git log` and `git blame` show history and who changed what; `git revert` undoes a commit safely.

10. Labs

Lab 1. Predict, then run. For each snippet, write down the output before running it:

```
x = [1, 2]; y = x; y += [3]; print(x)
print(0.1 + 0.2 == 0.3, round(0.1 + 0.2, 10) == 0.3)
def f(item, items=[]):
    items.append(item); return items
print(f(1), f(2))
print(sorted(["banana", "apple", "Cherry"]), sorted(["banana", "apple", "Cherry"],
key=str.lower))
```

Lab 2. Feel Big-O. Time the list vs set membership example from Section 5 with 20,000 items, then 40,000. Notice that doubling the input roughly quadruples the list version's time and barely changes the set version.

Lab 3. Extend the rubric example. Add a third rubric class (e.g. average sentence length) to the Section 4 code without changing `score_all`. That's polymorphism and the open/closed principle in practice.

Lab 4. Review AI code. Ask an AI tool to write a function that takes a list of session records and returns each user's best score in the last 30 days. Review it with the Section 9 checklist. Find at least two improvements (edge cases, complexity, naming, tests), and ask for them precisely.

Lab 5. Recursion. Write a function that finds the deepest nesting level of a JSON-like structure of dicts and lists. Then add memoisation to a recursive Fibonacci and compare `fib(32)` with and without it.

11. Self-check questions

1. **Why did changing `b` also change `a`?** Both names refer to the same mutable list; assignment doesn't copy.
2. **Why shouldn't money be stored as a float?** Floats are binary approximations; small rounding errors accumulate. Use integer paise or a decimal type.
3. **What's wrong with `except: pass`?** It hides every failure, so bugs become silent wrong results.
4. **Encapsulation, inheritance, polymorphism in a line each?** Hide internals behind a small interface; extend a class to reuse and override behaviour; different types answer the same method call in their own way.
5. **Why prefer composition to deep inheritance?** Deep hierarchies tie classes together tightly, so changes ripple unpredictably; composing small parts stays flexible.
6. **When would you use a dict instead of a list?** When you look things up by key, count, group or de-duplicate; lookups are O(1) instead of O(n).
7. **Spot the $O(n^2)$:** `[x for x in a if x in b]` with b a list. Fix by converting b to a set.
8. **What does O(log n) feel like?** A million items takes about 20 steps; binary search and database indexes.
9. **Why can an O(n) loop still be slow?** If each iteration does I/O (a query or API call), constants dominate; batch the I/O.
10. **What must every recursive function have?** A base case, and progress toward it on each call.
11. **What is memoisation?** Caching results of function calls so repeated subproblems aren't recomputed; the heart of dynamic programming.
12. **BFS vs DFS?** BFS explores level by level (shortest paths in unweighted graphs); DFS goes deep first (dependencies, exhaustive search).
13. **Name three red flags in AI-written code.** Hallucinated APIs, swallowed errors, and tests changed to pass instead of fixing the code.
14. **Where do you start reading unfamiliar code?** At the entry point for one real request, plus the tests.

12. Resources

Check these are still available before committing time.

- **Harvard CS50** (free online). The best broad introduction to programming, data structures and the web.
- **Al Sweigart, *Automate the Boring Stuff with Python*** (free online). Practical Python for real tasks.
- **Aditya Bhargava, *Grokking Algorithms*.** Illustrated, gentle algorithms and Big-O.
- **Big-O Cheat Sheet** (bigocheatsheet.com) for the costs of common structures and operations.
- **Python's official tutorial** and the **MDN JavaScript Guide** for language reference.
- **Google's Engineering Practices guide on code review** (free on GitHub) for how reviewers think.
- **Pro Git** (free at git-scm.com/book) for version control.